

Computational Extension of OEIS Sequence A224540: New Terms Through $a(28)$ via a Truncated Inverse Collatz Tree

Carlo Corti
Independent researcher
`carlo.corti@outlook.com`

June 2026

Abstract

OEIS A224540 counts the positive integers k whose Collatz trajectory, stopped at the first occurrence of 1, never exceeds 3^n . The public OEIS b-file previously contained terms through $a(21)$. This paper reports a computational extension through $a(28)$, with new values

$$\begin{aligned}a(22) &= 12406199367, & a(23) &= 37216010685, \\a(24) &= 111651745638, & a(25) &= 334951946950, \\a(26) &= 1004855123675, & a(27) &= 3014552251123, \\a(28) &= 9043690122807.\end{aligned}$$

The computation used an iterative depth-first traversal of the inverse Collatz tree rooted at the terminal value 1, truncated by the inclusive bound $M = 3^n$. The largest terms were produced by segmented subtree campaigns and checked by an independent artifact-level validator that reparsed the result files, recomputed powers 3^n , verified segment coverage, and recomputed aggregate sums. The independent validation was not a second full traversal of all inverse subtrees for $n = 28$.

2020 Mathematics Subject Classification. Primary 11Y55; Secondary 11B83, 11Y35.

Key words and phrases. Collatz iteration, $3x + 1$ problem, integer sequence, OEIS, inverse tree, computational number theory.

1 Introduction

Let T denote the usual Collatz map,

$$T(k) = \begin{cases} k/2, & k \equiv 0 \pmod{2}, \\ 3k + 1, & k \equiv 1 \pmod{2}. \end{cases}$$

For a starting value k whose forward orbit reaches 1, the stopped trajectory is the finite sequence $k, T(k), T^2(k), \dots, 1$, truncated at the first occurrence of 1. OEIS A224540 is the sequence

$$a(n) = \#\{k \geq 1 : \text{the trajectory of } k, \text{ stopped at the first } 1, \text{ never exceeds } 3^n\}.$$

Equivalently, $a(n)$ counts the positive starting values k whose stopped trajectory maximum is at most 3^n . The inequality is inclusive. For $k = 1$, the stopped trajectory is the one-term trajectory $[1]$, so 1 is counted for every $n \geq 0$.

For example, for $n = 3$, the bound is $3^3 = 27$, and the twelve values listed in the OEIS entry are

$$1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24.$$

The local OEIS snapshot used for this project had offset 0, 2, listed terms through $a(21)$, and credited T. D. Noe as the author of the entry, with $a(20)$ and $a(21)$ supplied by Donovan Johnson [1].

The direct definition suggests scanning all $k \leq 3^n$ and computing trajectory maxima. The production computation instead enumerates exactly the accepted values by traversing the inverse Collatz tree from the terminal root 1, with the same ceiling $M = 3^n$. This gives a simple finite graph problem for each n , avoids a visited set, and keeps memory usage governed by an explicit DFS stack rather than by the size of the accepted set.

The contribution of this paper is the extension of the b-file through $n = 28$, together with the finite computation and validation artifacts that support it. The result is computational: it does not assert anything about the Collatz conjecture beyond the finite stopped trajectories represented in the truncated inverse tree. Standard background on the $3x + 1$ problem is given by Lagarias [2, 3].

2 Definition and inverse-tree framework

Definition 1. For $M \geq 1$, let S_M be the smallest set of positive integers obtained as follows. The root 1 belongs to S_M . If $x \in S_M$, then the following inverse predecessors are admitted when they satisfy the displayed conditions:

$$2x \quad \text{if } 2x \leq M,$$

and

$$\frac{x-1}{3} \quad \text{if } x \equiv 1 \pmod{3}, \quad \frac{x-1}{3} \text{ is odd}, \quad \frac{x-1}{3} > 1, \quad \frac{x-1}{3} \leq M.$$

Remark 2. The condition $(x-1)/3 > 1$ is part of the stopped-at-1 convention. Without it, the odd predecessor 1 of 4 would create the trivial cycle 1, 4, 2, 1 inside the inverse construction. In A224540 the trajectory is stopped when 1 is first reached, so 1 is a terminal root, not a node whose outgoing Collatz step is followed again.

Proposition 3 (Truncated inverse-tree equivalence). *For every $M \geq 1$, S_M is exactly the set of positive integers $k \leq M$ whose Collatz trajectory, stopped at the first occurrence of 1, never exceeds M .*

Proof. If y is admitted as an inverse predecessor of x , then $T(y) = x$: this is immediate for the doubled predecessor $y = 2x$, and for the odd predecessor $y = (x-1)/3$ it follows from the imposed congruence and parity conditions. Therefore every node generated from the root has a finite forward path to 1, obtained by reversing its generation path. Every node on that path was admitted only after the bound check $y \leq M$. Hence every generated node satisfies the defining property.

Conversely, let $k \leq M$ have a stopped trajectory

$$k, T(k), T^2(k), \dots, 1$$

all of whose entries are at most M . Reversing this finite path starts at 1. Each reversed step is either $x \mapsto 2x$, for an even predecessor, or $x \mapsto (x-1)/3$, for an odd predecessor. The predecessor is odd in the second case; moreover, if that predecessor is y , then $x = T(y) = 3y + 1$, so $x \equiv 1 \pmod{3}$. It remains at most M by hypothesis and is greater than 1 because the value 1 occurs only

as the terminal element of the stopped trajectory. Thus every element of the stopped trajectory, in particular k , is generated in S_M .

The next Collatz iterate of any positive integer is unique. Therefore a generated node $y \neq 1$ cannot enter the rooted inverse construction under two different parents x , where the parent is the forward image $T(y) = x$ in the rooted tree. The rule excluding the odd predecessor 1 of 4 removes the only terminal-cycle re-entry relevant to the stopped convention. Consequently the inverse traversal counts each accepted starting value exactly once. $\square$

Taking $M = 3^n$, Proposition 3 gives

$$a(n) = \#S_{3^n}.$$

This is the mathematical basis for the production program.

3 Production traversal and segmentation

The production source is a C17 program, stored as `src/a224540.c` in the raw GitHub-ready package [4]. For a fixed $M = 3^n$, it performs an iterative DFS over the inverse tree:

```

initialize stack with 1;
while the stack is nonempty, pop  $x$  and count it;
push  $2x$  if  $2x \leq M$ ;
push  $(x - 1)/3$  under the odd-predecessor conditions above.
```

The implementation forms the doubling test as $x \leq M/2$, so that $2x$ is not computed before the bound check. It stores states and counts in `uint64_t` within the range used here, and uses `__uint128_t` for decimal parsing, powers of 3, and overflow checks. The project numeric ledger records that $3^{28} = 22876792454961$, far below the `uint64_t` limit; the code rejects powers that do not fit.

The program has several modes relevant to reproducibility. A replay mode recomputes the inverse-tree counts for the compiled known prefix $n = 0, \dots, 21$ and compares them with the OEIS-prefix rows in the b-file. A range mode produced $a(22)$ through $a(25)$. For the larger terms, the traversal was split at a fixed depth into a finite frontier, and contiguous intervals of frontier indices were processed as independent segment jobs. Aggregation then added the prefix count and the segment subtree counts, after checking that segment intervals covered the whole frontier without gaps or overlaps. Table 1 summarizes the production campaigns.

For segmentation, the depth of the root 1 is 0, and the depth of a node is the number of inverse steps from the root. For a chosen depth d , the frontier is the ordered list of nodes first encountered at exactly depth d during the same deterministic iterative DFS used by the program. The child-generation order is the doubled child first, then the odd predecessor when it exists; the stack is last-in, first-out, so when both children exist the odd predecessor is visited before the doubled child. The frontier order is this pop/visit order, not merely the textual order of the push operations. The prefix count is the number of nodes at depths $< d$. Frontier indices are zero-based. A segment with interval $[i, j]$ processes the complete subtrees rooted at frontier indices $i, i + 1, \dots, j$, inclusive. Since Proposition 3 gives a rooted tree, these segment subtrees are disjoint, and the union of all frontier intervals together with the prefix nodes is the full truncated tree.

Segmentation was used for resumability and auditability, not as a load balancing claim. For $n = 28$, the frontier at depth 24 contained only 179 roots, but the subtrees rooted at those frontier nodes had very different sizes. The initial equal-size plan was refined by splitting one adjacent

frontier region, so the final aggregate uses fourteen segment files; the complete ledger is Table 3. Two segment files account for most of the wall-clock elapsed time: frontier indices 0–14 recorded 4021158357110 counted nodes in 11951.82 seconds, while indices 45–59 recorded 4064690194873 counted nodes in 12149.51 seconds. Other segments with similar numbers of roots were much smaller; for example, indices 165–178 recorded 4732930237 nodes in 14.27 seconds. This imbalance is an operational property of the truncated inverse Collatz tree, not an aggregation error.

The production environment for the recorded local campaign was an AMD Ryzen 9 7940HS system with 16 logical CPUs and 64 GB DDR5 installed RAM, running Linux Mint 22.3 on x86-64. The underlying distribution base reported by kernel or package tooling is not used as the operating-system description for this paper. The current package builds with GCC 13.3.0 as a C17 program; the release Makefile uses the optimization flags `-O3`, `-march=native`, `-flto`, `-funroll-loops`, `-fomit-frame-pointer`, and `-DNDEBUG`, plus warning flags. For checked intermediate arithmetic, the implementation uses the GCC/Clang extension `__uint128_t` under this documented GCC build environment. The validation target recorded in the package was

```
make -C src all validate sanitize RUN_ID=20260630Tbfile_extended_final.
```

Here `validate` is the full validation target; in the public Makefile, `test` is an alias for `validate`. It runs the production-program self-tests, CLI negative tests, known-prefix replay, calibration, small segmented-versus-monolithic checks, aggregate adversarial tests, the independent top-down check through $n = 14$, and local checksum generation. The separate artifact-level validator is the Python script described below.

n	$M = 3^n$	mode	depth	segments	elapsed seconds
22–25	3^{22} – 3^{25}	range	–	0	1634.34
26	2541865828329	segmented	18	1	3021.58
27	7625597484987	segmented	20	2	9039.15
28	22876792454961	segmented	24	14	26987.87

Table 1: Production campaign summary from the run manifest TSV, with elapsed times rounded to two decimals. The first row is a single range campaign for $n = 22, \dots, 25$. Elapsed times are wall-clock measurements from the run manifests; rounded segment displays need not sum exactly to a separately recorded rounded campaign total.

4 Validation and certification boundary

The validation record has three layers.

First, the production program contains local child-rule tests, branch-order invariance tests, and segmented-versus-monolithic checks on small instances. Here branch-order invariance means that small counts agree under the supported child-order variants, while segmented-versus-monolithic checks compare a small frontier partition with a single unsegmented count. The Makefile target `make -C src test`, an alias of `make -C src validate`, runs CLI tests, replay of the compiled known prefix, a small independent top-down check through $n = 14$, segmented/monolithic checks at $n = 14$ and depth 8, and aggregate adversarial checks. The top-down check is a separate Python enumerator used only at small scale. The target `make -C src sanitize` builds and runs an AddressSanitizer/UndefinedBehaviorSanitizer smoke validation.

Second, the known OEIS prefix $a(0), \dots, a(21)$ was replayed. In the published raw package, `data/certified_terms.tsv` records these terms as coming from the OEIS b-file replay oracle, while $a(22), \dots, a(25)$ are from the range campaign and $a(26), a(27), a(28)$ are from segmented aggregate campaigns.

Third, the script `tools/a224540_validate_new_results.py` performs an independent artifact-level validation. It does not call the production binary. It parses the range and aggregate TSV files, validates that `data/b224540.txt` is contiguous for $n = 0, \dots, 28$, recomputes the powers 3^n , reconstructs the finite frontier summaries for segmented campaigns, checks complete segment coverage, and recomputes aggregate sums. For segmented campaigns, complete coverage means that the recorded frontier intervals partition the zero-based frontier without gaps or overlaps; the validator also checks that the aggregate total is the prefix count plus the recorded segment subtree counts. Its recorded verdict is PASS. The status `AGGREGATED_NO_ORACLE` denotes an aggregate reconstructed from segment artifacts without a separate external oracle for that large term.

n	depth	prefix	frontier	segments	total
26	18	168	44	1	1004855123675
27	20	270	72	2	3014552251123
28	24	689	179	14	9043690122807

Table 2: Segmented campaign checks recomputed by the independent artifact-level validator. The totals include the prefix count and all segment subtree counts.

frontier interval	roots	subtree count	elapsed seconds
0–14	15	4021158357110	11951.82
15–15	1	9437625941	28.28
16–20	5	113738272773	345.38
21–29	9	30709962147	93.37
30–44	15	133937400225	406.50
45–59	15	4064690194873	12149.51
60–74	15	89293833363	265.38
75–89	15	19101684797	57.79
90–104	15	179424181121	537.10
105–119	15	32810854681	98.37
120–134	15	179352408944	543.57
135–149	15	157417194040	472.74
150–164	15	7885221866	23.78
165–178	14	4732930237	14.27

Table 3: Complete $n = 28$ segment ledger from the campaign TSVs, with elapsed times rounded to two decimals for readability. Together with the prefix count 689, the subtree counts sum to $a(28) = 9043690122807$.

The exhaustiveness of the finite enumeration for each n rests on the production kernel and the recorded campaign manifests: every node of the corresponding truncated inverse tree is counted once by the traversal or by a complete frontier partition. The independent validator certifies the consistency of the produced artifacts and the segment aggregation, but it is not a second production-scale traversal of all inverse subtrees for $n = 28$. This distinction is part of the certification boundary.

More explicitly, the independent validator checks file syntax, the contiguous b-file through $n = 28$, recomputed powers 3^n , consistency of the range and aggregate TSVs, complete segment coverage, and aggregate sums. It does not prove the absence of a systematic defect in the production traversal by rerunning the full computation with a distinct implementation. The supporting evidence for that part is the reviewed C17 kernel, replay of the known OEIS prefix, small independent top-down checks, sanitizer runs, segmented versus monolithic tests on small instances, adversarial aggregate tests, and the recorded production manifests.

5 Results

Theorem 4 (Artifact-checked computational extension through $a(28)$). *Subject to the correct execution of the reviewed production kernel and the recorded campaign manifests in the raw package, OEIS A224540 has the following values for $n = 22, \dots, 28$:*

n	$a(n)$
22	12406199367
23	37216010685
24	111651745638
25	334951946950
26	1004855123675
27	3014552251123
28	9043690122807

Together with the replayed OEIS prefix, the b-file is extended contiguously through $n = 28$.

Proof. For each n , Proposition 3 reduces $a(n)$ to the finite count of the truncated inverse tree S_{3^n} , with the inclusive bound $x \leq 3^n$ and the terminal-root convention at 1. The production program implements exactly these inverse-child rules and counts the generated nodes by DFS.

For $n = 22, \dots, 25$, the range campaign TSV `validation/campaigns/a22_a25/a22_a25.tsv` records the four counts in the theorem with status `COMPUTED`. For $n = 26, 27, 28$, the aggregate TSVs under `validation/campaigns/` record the segmented totals in Table 2 with status `AGGREGATED_NO_ORACLE`. The aggregate records contain complete frontier coverage data, and the independent validator recomputed the corresponding powers, prefixes, frontiers, segment coverage, and sums. The same validator checked that the extended b-file agrees with the result table for all $n = 0, \dots, 28$. Therefore the recorded finite computations give the displayed extension. $\square$

Corollary 5. *The OEIS-style b-file `data/b224540.txt` in the raw package is a contiguous table of $n, a(n)$ for $0 \leq n \leq 28$, with the seven new terms listed in Theorem 4. Table 4 summarizes the certification source for each part of the extended table.*

6 Conclusion

The computation extends A224540 from the previously recorded $a(21)$ to $a(28)$. The main mathematical point is that the stopped-at-1 Collatz condition is exactly represented by a finite inverse tree rooted at terminal 1, provided the predecessor 1 of 4 is excluded and the bound $x \leq 3^n$ is treated inclusively. The largest reported term,

$$a(28) = 9043690122807$$

n	$a(n)$	source in package
0–21	OEIS prefix through 4135278456	b-file replay oracle
22	12406199367	range campaign
23	37216010685	range campaign
24	111651745638	range campaign
25	334951946950	range campaign
26	1004855123675	segmented aggregate
27	3014552251123	segmented aggregate
28	9043690122807	segmented aggregate

Table 4: Certification sources summarized from the certified-terms and result TSV files.

comes from a segmented traversal of the truncated tree at $M = 3^{28} = 22876792454961$. The validation artifacts support the b-file extension through $n = 28$, with the explicit limitation that the independent validator checks artifact consistency and aggregation rather than repeating a second full production-scale traversal.

Data availability

The code, b-file, result tables, campaign TSVs, manifests, validation summaries, and independent artifact-level validator are part of the raw GitHub-ready package and are intended for inclusion in the project repository and archive listed in the References when those public artifacts are finalized. The essential reproducibility artifacts are `src/a224540.c`, `src/Makefile`, `data/b224540.txt`, `data/certified_terms.tsv`, `results/A224540_terms.tsv`, `validation/run_manifest.tsv`, the `validation/campaigns/` subtree, and `tools/a224540_validate_new_results.py`. The canonical OEIS-style b-file in the package is `data/b224540.txt`; result tables and certification ledgers are cross-checking artifacts, not competing b-files.

The GitHub URL and Zenodo DOI in the reference below are publication metadata for the final public release. They are intentionally provisional in this pre-release manuscript and must be updated only after the repository and Zenodo archive are officially published. Until that publication step occurs, their provisional status is a final-release packaging item, not a mathematical or computational defect in the draft.

AI use disclosure

Large language models were used as research and drafting assistants during the A224540 workflow, including OpenAI Codex and external LLM-based review systems from the ChatGPT, Claude, DeepSeek, Gemini, Grok, Kimi, Qwen, MiniMax and Z families. They are named at family level where exact model-version metadata were not consistently available in the local review artifacts.

Mathematical analysis. LLM assistance helped organize the stopped-trajectory convention, the inverse-tree equivalence, the terminal-root edge case, and the certification boundary. The mathematical claims used in this manuscript were checked against the production source, the OEIS snapshot, and the validation artifacts.

Algorithm and code. LLM tools assisted with code review, validation planning, and manuscript-facing explanations of the C17 implementation. The reported values do not depend on prose generated by an AI system; they are taken from the recorded production TSVs, b-file, manifest, and

independent artifact-level validator.

Manuscript. LLM assistance was used while drafting and reviewing this TeX manuscript. Numerical statements in the manuscript were cross-checked against the local package files before inclusion.

The author takes full scientific responsibility for the mathematical statements, algorithm, implementation, numerical results, validation boundaries, and conclusions of this paper.

References

- [1] OEIS Foundation Inc., *Sequence A224540: Number of numbers k such that all terms of the Collatz $(3x + 1)$ iteration of k are $\leq 3^n$* , The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A224540>. Local project snapshot accessed June 30, 2026 listed offset 0, 2, terms through $a(21)$, author T. D. Noe, Apr. 24, 2013, and extension $a(20)$ – $a(21)$ by Donovan Johnson, Jun. 5, 2013.
- [2] J. C. Lagarias, *The $3x + 1$ problem and its generalizations*, American Mathematical Monthly **92** (1985), no. 1, 3–23. <https://doi.org/10.1080/00029890.1985.11971528>.
- [3] J. C. Lagarias, editor, *The Ultimate Challenge: The $3x + 1$ Problem*, American Mathematical Society, Providence, RI, 2010. ISBN-13: 9780821849408; ISBN-10: 0821849409.
- [4] C. Corti, *A224540: Computational extension through $a(28)$ by truncated inverse Collatz-tree traversal*, raw GitHub-ready package and planned Zenodo archive, 2026. <https://github.com/carcorti/A224540>. Zenodo DOI placeholder: 10.5281/zenodo.xxxxxxxx, to be replaced after archive publication.